第二阶段：与 LangChain 及 LangGraph 的区别

学习目标：理解 DeepAgents 在 LangChain 生态系统中的定位，以及它与 LangChain、LangGraph 的区别和联系

目 录目 录

一、技术定位概述

二、技术定位对比

三、层次关系与角色分工

四、本章小结

一、技术定位概述

1.1 整体定位

框架	定位	适用场景
LangChain	基础组件库	需要自定义提示与工具的基础智能体搭建
LangGraph	编排引擎	构建复杂的多智能体系统与工作流
DeepAgents	应用框架	快速实现 AutoGPT 类应用，省去底层开发

1.2 本质理解

DeepAgents 本质上是基于 LangChain 的深度模式封装——

- 它并非替代 LangChain 或 LangGraph
- 而是将其常用抽象与运行时封装为开箱即用的组件
- 可视为一层"开发加速器"

二、技术定位对比

2.1 核心特性对比表

特性	LangChain	LangGraph	DeepAgents
层级	基础组件库 (Foundation)	编排引擎 (Orchestration)	应用框架 (Application Framework)
核心抽象	Chain Runnable Tool	StateGraph Node Edge	DeepAgent Middleware Backend
灵活性	极高 (积木块)	高 (自定义图结构)	中 (约定优于配置)
开箱即用	低 (需自行组装)	中 (需定义图逻辑)	高 (内置规划、文件系统、子代理)
适用对象	库开发者 底层构建	复杂流程 序列化开发者	应用开发者 企业级解决方案

2.2 详细解读

LangChain：积木块

定位：基础组件库 (Foundation)

核心抽象：Chain, Runnable, Tool

特点：提供 Prompt（提示词）、Models（模型）、Tools（工具）等积木；灵活性极高，但需要开发者自行组装；适合需要深度定制的场景

适用对象：库开发者、底层构建者

LangGraph：地基+连接

定位：编排引擎 (Orchestration)

核心抽象：StateGraph, Node, Edge

特点：提供 State（状态）、Nodes（节点）、Edges（边）等地基和连接逻辑；适合构建复杂的多智能体系统；需要开发者定义图逻辑

适用对象：复杂流程序列化开发者

DeepAgents：成品框架

定位：应用框架 (Application Framework)

核心抽象：DeepAgent, Middleware, Backend

特点：预设了最佳实践（规划、文件系统、子智能体）；隐藏了底层的图构建细节；开箱即用，快速构建类似 OpenAI Deep Research 或 Claude Code 的应用

适用对象：应用开发者、企业级解决方案

三、层次关系与角色分工

3.1 层次关系图

```
┌──────────────────────────────────────────────────────────┐
│                        DeepAgents                        │
│                (成品级框架 - 应用层)                      │
│  ┌──────────────────────────────────────────────────┐    │
│  │ 预设最佳实践：规划、文件系统、子智能体            │    │
│  │ 高级 API：create_deep_agent()                     │    │
│  └──────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│                        LangGraph                         │
│                (编排引擎 - 框架层)                       │
│  ┌──────────────────────────────────────────────────┐    │
│  │ State：状态管理                                   │    │
│  │ Nodes：节点定义                                   │    │
│  │ Edges：连接逻辑                                   │    │
│  └──────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│                        LangChain                         │
│                (基础组件库 - 库层)                       │
│  ┌──────────────────────────────────────────────────┐    │
│  │ Prompt：提示词管理                                │    │
│  │ Models：模型接口                                  │    │
│  │ Tools：工具定义                                   │    │
│  └──────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────┘
```

3.2 角色分工

层次	框架	角色	贡献
库层	LangChain	积木提供者	提供基础的提示词、模型、工具组件
框架层	LangGraph	地基构建者	提供状态管理和流程编排能力
应用层	DeepAgents	成品制造者	封装最佳实践，提供开

			箱即用的解决方案

3.3 协同关系

LangChain（积木）——→ LangGraph（地基+连接）——→ DeepAgents（成品框架）

LangChain：提供 Prompt，Models，Tools 等积木

LangGraph：提供 State，Nodes，Edges 等地基和连接逻辑

DeepAgents：LangGraph 的"最佳实践实现"，使用 LangGraph 管理状态和循环，隐藏底层图构建细节

四、本章小结

核心要点

1. **技术定位**

- LangChain：基础组件库
- LangGraph：编排引擎
- DeepAgents：应用框架

2. **核心区别**

- LangChain 灵活性最高，但开箱即用性低
- LangGraph 适合复杂工作流，但需要定义图逻辑
- DeepAgents 开箱即用性最高，但灵活性适中

3. **层次关系**

LangChain → LangGraph → DeepAgents，层层封装，逐步简化开发复杂度

4. **本质理解**

DeepAgents 是 LangChain/LangGraph 的"开发加速器"，并非替代关系，而是互补关系

课后思考

1. 你的项目更适合使用 LangChain、LangGraph 还是 DeepAgents？
2. DeepAgents 的"约定优于配置"理念对你的开发效率有什么影响？
3. 在什么场景下你会选择直接使用 LangGraph 而不是 DeepAgents？

本课件基于《DeepAgents_框架介绍与应用实践》第二阶段内容整理